CSE102 Assignment 9

Objective

This assignment aims to implement a complete pipeline that reads question-answer pairs from a text file, encodes them using character-based one-hot encoding, and writes the encoded embeddings to a structured output file along with metadata.

Summary of Implementation

The program performs the following main tasks, exactly as described in the assignment specification:

1. Loading the Dataset

Questions and answers are read from database.txt. The program detects lines prefixed with Q: and A:.

```
if (strncmp(line, QUESTION_PREFIX, strlen(QUESTION_PREFIX)) == 0) {
    strcpy(questions[count], line + strlen(QUESTION_PREFIX));
} else if (strncmp(line, ANSWER_PREFIX, strlen(ANSWER_PREFIX)) == 0) {
    strcpy(answers[count], line + strlen(ANSWER_PREFIX));
    count++;
}
```

This ensures that each pair is correctly stored in questions[i] and answers[i]

2. Unique Character Extraction

All unique characters across questions and answers are stored in the allChars[] array to build the one-hot encoding space.

```
if (!strchr(allChars, text[t][j])) {
    allChars[charCount++] = text[t][j];
}
```

This character set determines the length of each one-hot vector (i.e., charCount).

3. Tokenization

Each sentence is split into words using strtok(), and the words are stored for further processing.

```c
char *token = strtok(sentence, " ");
while (token) {
    strcpy(words[(*wordCount)++], token);
    token = strtok(NULL, " ");
}
```

This tokenized representation enables encoding at the word level.

4. One-Hot Encoding (Character Level)

Every character in a word is encoded into a vector of length equal to the total number of unique characters. A single 1 is placed at the character's corresponding index.

```c
void oneHotChar(char ch, int charCount, char allChars[], int *vec) {
    for (int i = 0; i < charCount; i++) vec[i] = 0;
    for (int i = 0; i < charCount; i++) {
        if (ch == allChars[i]) {
            vec[i] = 1;
            break;
        }
    }
}
```

5. Left-Padded Word Embedding

If a word is shorter than the maximum word length, it is left-padded with zero-vectors.

```c
int pad = maxWordLen - strlen(word);
for (int i = 0; i < pad * charCount; i++) output[i] = 0;
oneHotChar(word[i], charCount, allChars, &output[(i + pad) * charCount]);
```

This satisfies the padding requirement from the assignment spec.

6.Sentence Vector Formation

Words are individually encoded and then concatenated to form the full sentence vector.

```c
vec[i * maxWordLen * charCount] = ... // word vector copied in sequence
```

The total embedding length of each sentence is stored and compared to compute Maximum Sentence Embedding Length.

7. Metadata and Output File Generation

The output is structured with metadata as required:

```
fprintf(f, "Maximum Sentence Embedding Length: %d\n", maxLen);
fprintf(f, "Dimension of Word Embeddings: %d\n", dim);
fprintf(f, "Number of Question-Answer Pairs: %d\n\n", pairCount);
fprintf(f, "#####\n");
```

Followed by the binary-encoded vectors:

```
fprintf(f, "Question: ");
for (int j = 0; j < qLens[i]; j++) fprintf(f, "%d", qVecs[i][j]);
```

Each pair is separated by "---" as required.

Output Example:

```
Maximum Sentence Embedding Length: 24120
Dimension of Word Embeddings: 1206
Number of Question-Answer Pairs: 141

#####
Question: 000000000000000000000000000000
Answer: 000000000000000000000000000000
---
Question: 000000000000000000000000000000
Answer: 000000000000000000000000000000
---
Question: 000000000000000000000000000000
Answer: 000000000000000000000000000000
---
Question: 000000000000000000000000000000
Answer: 000000000000000000000000000000
```

Conclusion

The implementation follows all instructions as stated in the assignment document. All encoding rules, formatting requirements, and modular coding standards are fully respected. The solution is efficient, readable, and extensible.